

基于结构特征与历史反馈的 IC3 归纳概括排序方法

摘 要

归纳概括通过逐文字删除缩减阻断子句，是影响 IC3/PDR 帧强化与收敛效率的关键环节。由于该过程采用贪心搜索，文字的尝试删除顺序可能使算法到达不同的局部最小子句。针对基于 SMT 的 IC3 实现中算术文字表示多样、固定删除顺序难以利用历史证明信息的问题，本文提出一种结合线性整数文字规范化、抽象语法树节点数和历史保留频率的自适应排序方法。该方法首先对量词消去生成的可处理算术文字进行规范化，以稳定后续结构度量与历史键；随后根据除法相关算子、历史反馈权重、抽象语法树节点数及原始位置构造字典式排序键，引导阻断子句的逐文字归纳概括。该方法不改变初始性与相对归纳性检查，仅调整候选删除文字的访问顺序，因而保持 IC3 原有的逻辑接受条件。本文在 Kind2/IC3QE 中实现该方法，并给出相应的形式化描述、实现分析与评估方案。

关键词：IC3/PDR；归纳概括；文字排序；算术规范化；历史反馈；软件模型检测

IC3 Inductive-Generalization Ordering Based on Structural Features and Historical Feedback

Abstract: Inductive generalization minimizes blocking clauses through iterative literal deletion and is a key operation affecting frame strengthening and convergence in IC3/PDR. Because the deletion procedure is greedy, different literal orders may lead to different locally minimal clauses. To address the diversity of arithmetic-literal representations and the inability of a fixed deletion order to exploit proof history in SMT-based IC3, this paper proposes an adaptive ordering method that combines linear-integer literal canonicalization, abstract-syntax-tree size, and decayed historical retention frequency. The method first canonicalizes supported arithmetic literals produced by quantifier elimination to stabilize structural measurements and history keys. It then constructs a lexicographic ordering key from the presence of division-related operators, historical feedback weight, abstract-syntax-tree size, and original position. The method changes only the order in which literal deletions are attempted; the initiality and relative-inductiveness checks remain unchanged. The approach is implemented in Kind2/IC3QE, together with a formal description, implementation analysis, and evaluation plan.

Keywords: IC3/PDR; inductive generalization; literal ordering; arithmetic canonicalization; historical feedback; software model checking

1 引言

本文关注 IC3/PDR 安全性模型检测中阻断子句的归纳概括阶段。IC3 在发现反例前缀不可达时，会学习新的阻断子句并将其加入帧序列；该过程通常通过逐文字删除的方式对候选子句进行概括。由于每一次文字删除都会改变后续初始性检查和相对归纳性检查面对的候选子句，文字尝试顺序会影响贪心概括的搜索路径，并可能产生不同的局部最小子句。本文以支持 Lustre [5] 的 SMT 模型检测工具 Kind2 [4] 及其 IC3QE 分析流程为实现基础。

1.1 研究动机

IC3 的归纳概括通常被实现为逐文字删除的贪心过程：算法依次尝试从阻断子句中移除某个文字，并通过初始性检查和相对归纳性检查判断删除是否安全。该过程在形式上仅依赖于逻辑判定结果，但在工程实现中，文字被尝试的先后顺序会决定贪心搜索首先访问哪些候选子句。若早期删除了某些关键文字，后续候选子句可能频繁无法通过归纳性检查；若优先删除局部冗余文字，则可能更快得到简洁且容易前推的阻断子句。因此，在不改变候选子句接受条件的前提下，设计低开销的文字排序策略具有直接意义。

另一方面，Kind2 等基于 SMT 的 IC3 实现面对的子句并不只包含布尔文字，还包含大量线性整数算术约束。此类约束存在丰富的语法等价形式，例如 $x + 1 = y$ 与 $y = x + 1$ 在语义上等价，但抽象语法树结构和内部项顺序不同。如果直接依据原始语法形式进行排序或统计，等价约束可能被视为不同对象，从而削弱历史信息的复用效果。由此产生了两个需要解决的问题：一是如何消除算术文字的非本质表达差异；二是如何利用文字结构和历史概括结果，为后续删除顺序提供稳定反馈。

现有研究已经表明，假设文字顺序和概括删除顺序会通过 SAT/SMT 求解器的传播、冲突分析、不可满足核以及满足赋值影响 IC3/PDR 或 CAR 的性能。例如，Intersection、Rotation、*i-good lemma*、CoreLocality 等策略分别从近期不可满足核、失败状态、历史成功引理或冲突文字中提取排序信号，用于引导求解器和概括过程生成更小、覆盖状态更多或更容易前推的引理。

本文方法面向线性整数算术场景，在不改变 IC3 逻辑判定条件的前提下优化归纳概括中的文字删除顺序。具体而言，方法首先对量词消去生成的可处理算术文字进行规范化，消除部分逻辑等价约束之间的非本质语法差异；随后使用抽象语法树节点数度量文字结构；最后结合是否包含除法或整数除法算子、历史反馈权重、抽象语法树节点数和原始位置构造字典式排序键。排序仅改变文字被尝试删除的先后，每个候选子句仍须通过第 2 节定义的初始性检查和相对归纳性检查。因此，该优化改变的是 IC3 归纳概括的搜索路径，而不是候选子句的逻辑接受条件。

本文的主要贡献可概括为以下三点：

1. 分析 IC3 逐文字归纳概括中的路径依赖现象，说明固定删除顺序可能影响概括结果、阻断子句覆盖范围以及帧序列收敛效率。
2. 提出一种结合表达式归一化、抽象语法树节点数和历史反馈权重的自适应文字删除排序方法，用于线性整数算术子句的归纳概括。
3. 在基于 SMT 的模型检测工具 Kind2/IC3QE 中实现所提出的自适应排序策略，并设计系统性的消融实验与基准测试方案，从验证时间、SMT 调用次数和概括子句质量三个方面评估不同排序信号的作用。

本文其余部分组织如下：第 2 节介绍 IC3/PDR、帧序列和阻断子句归纳概括，并综述相关工作；第 3 节通过具体模型说明文字删除顺序的路径依赖性；随后给出本文方法、实现细节及复杂度分析；最后讨论实验设计、适用边界并总结全文。

2 背景知识与相关工作

2.1 IC3/PDR 与归纳概括

设状态变量集合为 X ，其下一状态副本为 X' 。安全性验证问题由转移系统 $M = (I, T)$ 和安全属性 P 构成，其中 $I(X)$ 表示初始状态集合， $T(X, X')$ 表示转移关系， $P(X)$ 表示安全状态集合。对于状态公式 $\varphi(X)$ ，记 $\varphi'(X')$ 为将其中每个状态变量替换为对应下一状态变量后得到的公式。

IC3/PDR [1, 2] 维护帧序列 $F_0, F_1, \dots, F_k$, 其中 $F_0 = I$, 且各帧满足安全性、单调性与相对归纳性约束:

$$F_i \Rightarrow P, \quad (1)$$

$$F_i \Rightarrow F_{i+1}, \quad (2)$$

$$F_i \wedge T \Rightarrow F'_{i+1}. \quad (3)$$

当相邻两帧达到不动点时, 该帧构成证明安全属性的归纳不变式。若帧中出现可到达属性违例的归纳反例 (counterexample to induction, CTI), 算法递归阻断其前驱, 并学习阻断子句以排除相应状态区域。

设 C 为待概括的阻断子句, $C_{\text{cand}} \subseteq C$ 为删除一个或多个文字后得到的候选子句。Kind2/IC3QE 仅在以下两个查询均不可满足时接受候选子句:

$$\text{SAT}(I \wedge \neg C_{\text{cand}}) = \text{false}, \quad (4)$$

$$\text{SAT}(P \wedge F_{i-1} \wedge C_{\text{cand}} \wedge T \wedge \neg C'_{\text{cand}}) = \text{false}. \quad (5)$$

式 (4) 保证候选子句不排除初始状态; 式 (5) 保证候选子句相对于前驱帧保持归纳有效。逐文字归纳概括依次构造候选子句并执行上述检查。一次成功删除会改变后续候选子句, 因此该过程通常只能获得依赖访问顺序的局部最小结果。

2.2 文字排序相关工作

IC3/PDR 在阻断反例和归纳概括过程中需要频繁调用 SAT/SMT 求解器。在逻辑上等价的情况下, 文字顺序仍可能改变求解器的传播与冲突分析过程, 进而影响不可满足核、前驱状态及概括子句的结构。现有研究中的“子句排序”主要涉及两个层面: 一是调整 SAT 调用中假设文字的输入顺序, 以引导求解器产生更小或更稳定的不可满足核; 二是调整归纳概括中文字的尝试删除顺序, 以引导贪心概括过程获得覆盖更多状态或更容易前推的子句。

Dureja 等人研究了假设文字顺序对 CAR 求解过程的影响, 提出 **Intersection** 和 **Rotation** 两种启发式策略 [6]。**Intersection** 将当前状态与最近一次不可满足核的公共文字排在假设序列前部, 以提高这些文字再次进入不可满足核的可能性; **Rotation** 则利用近期失败状态的公共文字调整顺序, 以增强搜索的多样性。该工作说明, 即使不修改 SAT 求解器内部算法, 仅通过重排假设文字也能改变不可满足核和满足赋值的形态。

Xia 等人从帧序列收敛的角度提出 *i*-good lemma 概念, 并通过 **branching** 和 **refer-skipping** 两类启发式引导 IC3/PDR 与 CAR 生成更容易前推的引理 [7]。其中, **branching** 根据变量在近期 *i*-good lemma 中的出现情况维护动态分数: 在 SAT 假设中优先排列高分变量对应的文字, 而在归纳概括中优先尝试删除低分文字, 从而倾向于保留近期可前推引理中频繁出现的变量。**refer-skipping** 则利用前一帧中已有的包含关系选取参考子句, 并在概括过程中跳过参考子句中的文字, 使新子句尽量保留已被验证的有效结构。该工作表明, 历史成功引理可以作为文字排序的反馈信号。

Seufert 等人对 PDR 中证明义务的概括问题进行了系统研究 [8]。该工作证明一般情况下的精确概括问题为 Π_2^P -完全, 并比较了三值仿真、**lifting**、**justification**、基于蕴含图的概括以及 QBF/MaxQBF 等方法的适用条件与概括能力。虽然该工作不专门讨论文字排序, 但其研究说明了在一般迁移关系上追求精确最优概括的计算代价较高, 因此工程实现通常依赖近似或贪心方法。在逐文字删除的贪心过程中, 尝试顺序会决定搜索路径及最终的局部最小结果, 从而为研究低开销的文字排序启发式提供了动机。

Hassan 等人从反例处理与文字删除策略的角度研究 IC3 中更强的归纳概括方法 [3], 说明概括质量会显著影响后续阻断与帧传播过程。该工作关注如何获得更强的概括结果, 本文则进一步考察固定贪心删除过程中文字访问顺序及算术文字表示对概括轨迹的影响。

Dong 等人进一步对 CAR 中的假设文字排序进行了系统分析 [9]。该工作以“不可满足核局部性”解释 Intersection 策略的有效性, 并提出 CoreLocality 策略, 根据近期多个不可满足核对更多假设文字进行分层排序。此外, 该工作还将触发冲突的文字赋予更高优先级, 并通过 Hybrid-CAR 在运行期间动态切换不同的排序配置。这些结果进一步证明, 排序策略的效果具有阶段性, 利用近期求解历史并动态调整顺序可以改善搜索效率。

表 1: IC3/PDR 及 CAR 中的子句与文字排序相关工作

相关工作	排序或概括对象	主要依据	主要目标
Dureja 等 [6]	SAT 假设文字	最近不可满足核与失败状态	生成更小或更稳定的不可满足核与满足赋值
Xia 等 [7]	SAT 假设文字与概括删除顺序	<i>i-good lemma</i> 变量分数与前驱帧参考子句	生成更容易前推的引理
Hassan 等 [3]	IC3 阻断子句的归纳概括	反例处理与概括策略	获得更强的阻断子句
Seufert 等 [8]	证明义务概括	迁移关系性质与多种精确/近似技术	提高概括强度并明确方法的适用条件
Dong 等 [9]	CAR 中的 SAT 假设文字	多个近期不可满足核、冲突文字与动态配置	增强不可满足核局部性并加速错误发现

总体而言, 现有工作主要围绕三类对象展开排序: SAT 调用中的假设文字、CAR 搜索中的失败状态相关文字, 以及 PDR/IC3 归纳概括中的候选删除文字。它们使用的反馈信号主要来自不可满足核、近期失败状态、冲突文字、历史成功引理或前一帧中的参考子句。这些信号能够改善布尔或 SAT 主导场景下的搜索路径, 但较少直接处理 SMT 归纳概括中算术文字的等价表示问题。

本文与上述工作的差别在于, 本文直接面向 IC3 逐文字归纳概括中的删除顺序, 而不是仅调整 SAT 假设序列。排序依据也不只来自求解历史, 还包含线性算术表达式归一化后的语法结构。通过在归纳概括前统一等价表达式, 并结合抽象语法树节点数与历史保留频率, 本文试图弥补现有方法对 SMT 算术文字内部结构关注不足的问题。

3 动机示例

归纳概括 (Inductive Generalization, IndGen) 通过将阻断子句 $\neg s$ 缩减为更小的归纳子句来加速 IC3 的收敛。给定第 i 帧上的 CTI 立方 s , IndGen 依次考察满足 $C_{\text{cand}} \subseteq \neg s$ 的候选子句 C_{cand} 。候选子句仅在同时满足下列初始性条件与相对归纳性条件时才被接受:

$$\text{SAT}(I \wedge \neg C_{\text{cand}}) = \text{false}, \quad (6)$$

$$\text{SAT}(P \wedge F_{i-1} \wedge C_{\text{cand}} \wedge T \wedge \neg C'_{\text{cand}}) = \text{false}. \quad (7)$$

式 (6) 保证概括后的子句不会排除任何初始状态; 式 (7) 保证该子句相对于前一帧仍然归纳有效。

标准实现通常按某个固定顺序逐一尝试删除 $\neg s$ 中的文字, 并调用 SMT 求解器检查删除后的候选子句是否仍满足上述两个条件。虽然每一步的接受条件是确定且可靠的, 但整个过程具有明显的路径依赖性: 一次成功删除会改变后续检查面对的候选子句, 因此同一个 CTI 可能在不同删

除顺序下收敛到不同的局部最小子句。这意味着文字顺序不仅影响最终子句的大小，也会影响其具体结构、状态覆盖范围以及后续帧序列的收敛路径。

为了展示这一现象，考虑图 1 中的 Lustre 程序。该程序同时定义一个 2 位 Gray 计数器和一个在边界 3 处复位的整数计数器，并验证二者输出是否同步。两个计数器结构不同，但安全属性 OK 要求其输出保持等价。

```
node greycounter () returns (out: bool);
var a, b: bool;
let
  a = false -> not pre b;
  b = false -> pre a;
  out = a and b;
tel

node intcounter (const max: int) returns (out: bool);
var t: int;
let
  t = 0 -> if pre t = max then 0 else pre t + 1;
  out = t = 2;
tel

node top (reset: bool) returns (OK, OK2: bool);
var b, d: bool;
let
  b = (restart greycounter every reset)();
  d = (restart intcounter every reset)(3);
  OK = b = d;
  OK2 = not d;
  --%PROPERTY OK;
tel
```

(a) reset_counters.lus 源程序

(b) 验证统计

	Fwd	Bwd
轮数 k	5	5
求解器调用	1740	1205
时间 (s)	1.000	0.637

(c) 不变式子句统计

	Fwd	Bwd
总数	20	14
不同子句	12	9
独有子句	7	4
共享不同子句	5	

图 1: 不同文字删除顺序下的 Lustre 示例及验证结果

程序中的 OK 是本示例实际验证的安全属性；OK2 仅为源基准保留的辅助输出，不参与此次属性检查。实验分别按照 Kind2 文字的确定性打印表示采用升序字典序（Fwd）和降序字典序（Bwd）作为文字删除顺序，并保持其他 IC3 参数不变。由于该安全属性在系统模型中恒成立，两种顺序最终都能得到归纳不变式并证明系统安全。然而，如图 1(b) 所示，二者虽然在相同的边界深度完成证明，Fwd 却执行了更多求解器调用，并在帧强化过程中消耗了更多时间。

这种时间差异也反映在最终不变式结构上。图 1(c) 表明，Fwd 产生的最终子句总数、不同子句数和顺序独有子句数均高于 Bwd；Bwd 在该实例上以更小的不变式完成了证明。进一步比较两组独有子句可以发现，两条轨迹保留了不同类型的阶段约束与算术同步关系。这些差异并非源于额外的语义规则，而是同一组归纳性检查在不同删除顺序下到达了不同的局部最小结果。为避免将 Kind2 展开后的内部变量与源程序变量混淆，本文不在此直接使用内部变量名表述独有不变式。

简单的文本顺序之所以可能产生影响，一个原因是 Lustre 展开以及 Kind2 内部命名会保留部分源程序命名空间信息。来自同一节点或同一通道的变量往往具有相关的文本名称，因此字典序删除可能聚集或分散描述同一组件交互的文字。在图 1 的例子中，这种偏置使某些顺序更早保留跨计数器同步子句，而反向顺序则倾向于保留不同的局部算术约束。该现象并不说明字典序具有普遍语义优势，而是说明文字删除顺序会和模型生成的变量命名发生可重复的交互。

因此，该示例揭示了本文工作的核心动机：固定删除顺序会使 IC3 只沿一条贪心概括轨迹前进，而不同的低开销确定性顺序可能到达不同的局部最小子句，并由此改变后续证明过程。本文进一步将这一观察推广到线性整数算术场景：在不改变初始性检查和相对归纳性检查的前提下，通

过表达式归一化、结构特征和历史反馈调整文字删除顺序，使归纳概括更倾向于保留对后续证明有效的子句结构。

需要指出的是，本文算法在历史权重相同时对抽象语法树（AST）节点数采取升序排列。在贪心删除机制下，这意味着算法将较早尝试删除节点数较少的文字。其设计意图是以低成本方式优先考察结构较简单的局部约束，并将结构较复杂的文字延后处理；但 AST 节点数并不直接刻画语义重要性或跨变量耦合强度。因此，该规则只作为历史反馈权重的辅助判据。文字能否被实际删除仍由初始性检查与相对归纳性检查共同决定，其有效性需要通过实验而非语法规则本身加以确认。

4 基于结构特征与历史反馈的 IC3 归纳概括方法

如前文所述，IC3 的逐文字归纳概括具有顺序敏感性。不同的文字尝试顺序可能产生不同的局部最小子句，进而影响所学习约束的状态覆盖范围和帧序列的收敛效率。

针对上述问题，本文提出一种基于结构特征与历史反馈的归纳概括优化方法。该方法包含两个相继执行的阶段。第一阶段位于 IC3QE 的量词消去之后、阻断子句构造之前：系统分别依据等式规范化开关和不等式规范化开关，对量词消去生成的可处理线性整数文字进行规范化。第二阶段位于阻断子句的归纳概括过程中：系统根据历史概括结果动态更新文字权重，并结合是否包含除法或整数除法算子、历史反馈权重、抽象语法树节点数和原始位置确定文字的尝试删除顺序。

该方法仅调整归纳概括阶段中文字的尝试顺序。对于每次文字删除，候选子句仍需通过初始性检查和相对归纳性检查。因此，该方法不改变候选子句的接受条件，也不影响所接受概括子句的逻辑有效性。

算法 1 给出了第二阶段的归纳概括流程。输入子句 C 已由 IC3QE 构造完成；若该子句源于启用规范化的量词消去路径，则其算术文字已在子句构造前完成规范化。当频率排序开关启用、前驱帧非空且子句包含至少两个文字时，算法根据是否包含除法或整数除法算子、历史反馈权重、抽象语法树节点数和原始位置构造排序键；否则沿用 Kind2 原有的文字顺序。完成排序后，算法线性扫描文字列表，并依次执行初始性检查和相对归纳性检查。

下面分别说明量词消去输出的算术文字规范化，以及归纳概括阶段的抽象语法树节点数评估与历史反馈排序策略。

4.1 量词消去输出的算术文字规范化策略

在 IC3QE 的具体实现中，系统首先调用量词消去过程生成描述归纳反例的文字列表；随后，在将这些文字取反并构造阻断子句之前，根据配置分别调用等式文字规范化和不等式文字规范化过程。同一逻辑约束可能具有多种语法表示。例如， $x + 1 = y$ 与 $y = x + 1$ 在语义上等价，但具有不同的语法树结构。若直接根据原始表达形式维护历史键或计算抽象语法树节点数，这类非本质差异可能影响后续排序结果。因此，规范化阶段仅对能够可靠线性化的整数比较文字生成确定性表示。

对于可线性化的整数表达式，首先将其表示为

$$e = \sum_{j=1}^n a_j t_j + c, \quad (8)$$

其中， t_j 表示整数项， a_j 为对应系数， c 为常数项。归一化过程合并相同项的系数、删除系数为 0 的项，并按照系统内部的确定性项序排列剩余项。

(1) 等式文字归一化。对于等式 $e_1 = e_2$ ，首先构造差值 $e_1 - e_2$ ，再将其转换为零右端形式：

$$e_1 - e_2 = 0. \quad (9)$$

算法 1 基于结构特征与历史反馈的自适应归纳概括

输入: 阻断子句 C ; 初始条件 I ; 安全属性 P ; 转移关系 T ; 前驱帧 F_{i-1}

输入/输出: 文字权重表 W

输出: 概括后的阻断子句 C_g

```

1  $L \leftarrow \text{Lits}(C)$ 
2 若  $F_{i-1} \neq \emptyset$  且  $|L| > 1$  则
3   对每个 文字  $l \in L$  执行
4        $d(l) \leftarrow \begin{cases} 0, & l \text{ 含除法或整数除法算子,} \\ 1, & \text{其他情况;} \end{cases}$  // 数值越小, 删除优先级越高
5        $w(l) \leftarrow W[l]$ ; 若  $l \notin W$ , 则令  $w(l) \leftarrow 0$ 
6        $a(l) \leftarrow \text{ASTNodes}(l)$  // 计算抽象语法树节点数
7        $p(l) \leftarrow l$  在  $L$  中的原始位置
8   按照  $\langle d(l), w(l), a(l), p(l) \rangle$  的字典序对  $L$  升序排序
9  $C_g \leftarrow C$ 
10 对每个 排序后的文字  $l \in L$  执行
11     若  $|\text{Lits}(C_g)| = 1$  则
12     | 终止循环 // 禁止将阻断子句概括为空子句
13      $C_{\text{cand}} \leftarrow C_g \setminus \{l\}$  // 构造删除当前文字后的候选子句
14     若  $\text{SAT}(I \wedge \neg C_{\text{cand}})$  则
15     | 继续 // 候选子句不满足初始性, 保留当前文字
16     若  $\text{SAT}(P \wedge F_{i-1} \wedge C_{\text{cand}} \wedge T \wedge \neg C'_{\text{cand}})$  则
17     | 继续 // 候选子句不满足相对归纳性, 保留当前文字
18      $C_g \leftarrow C_{\text{cand}}$  // 两项检查均不可满足, 接受本次删除
19 若  $C_g \neq C$  则
20     对每个 权重表中已有的文字  $l \in \text{dom}(W)$  执行
21     |  $W[l] \leftarrow \alpha W[l]$  // 衰减较早的历史反馈
22     对每个 概括后保留的文字  $l \in \text{Lits}(C_g)$  执行
23     |  $W[l] \leftarrow W[l] + 1$  // 增强近期关键文字的权重
24 返回  $C_g$ 

```

例如, $x + 1 = y$ 被转换为 $x - y + 1 = 0$ 。对于变量与常数之间的简单等式, 如 $x = 3$, 保留其原有形式。对于含有除法、整数除法、量词或 `let` 表达式的等式, 则不执行线性归一化, 以避免改变语义敏感结构。

(2) 不等式文字归一化。对于变量与常数之间的简单不等式, 如 $x < 3$, 系统保留其原有比较形式; 若该类文字带有外层否定, 如 $\neg(x > 3)$, 其否定结构也保持不变。对于其余可线性化的不等式, 系统首先消除外层否定并统一比较方向, 再将其改写为零右端形式。例如, $e_1 < e_2$ 被转换为 $e_2 - e_1 > 0$, 而 $\neg(e_1 > e_2)$ 被转换为 $e_2 - e_1 \geq 0$ 。完成移项后, 若线性表达式中按照确定性项序排列的首个非零系数为负, 则将整个表达式乘以 -1 , 并同步翻转比较符号。该操作能够进一步消除整体符号差异。对于仅含常量的比较表达式, 则直接对其进行常量求值以确定真值。

表 2 汇总了本文涉及的主要归一化操作。

表 2: 量词消去输出中算术文字的规范化操作汇总

操作类型	原始形式	归一化形式	作用
线性项收集	$x + 1 - y$	$x - y + 1$	合并同类项, 统一线性结构
零系数消除	$x - x + y$	y	删除无效变量项
常量折叠	$3 < 5$	true	对常量比较式进行求值
等式移项	$e_1 = e_2$	$e_1 - e_2 = 0$	将等式统一为零右端形式
简单等式保留	$x = c$	$x = c$	保留变量-常数约束的直观结构
等式保守过滤	含除法、量词等复杂算子	保持原式	保持原有语义结构, 规避非等价变换
简单不等式保留	$x < c$ 或 $\neg(x > c)$	保持原式	保留变量-常数边界的直观结构
大于归一化	$e_1 > e_2$	$e_1 - e_2 > 0$	统一不等式右端
大于等于归一化	$e_1 \geq e_2$	$e_1 - e_2 \geq 0$	统一不等式右端
小于归一化	$e_1 < e_2$	$e_2 - e_1 > 0$	统一比较方向
小于等于归一化	$e_1 \leq e_2$	$e_2 - e_1 \geq 0$	统一比较方向
否定大于	$\neg(e_1 > e_2)$	$e_2 - e_1 \geq 0$	消除否定并保持等价
否定大于等于	$\neg(e_1 \geq e_2)$	$e_2 - e_1 > 0$	消除否定并调整边界
否定小于	$\neg(e_1 < e_2)$	$e_1 - e_2 \geq 0$	消除否定并保持等价
否定小于等于	$\neg(e_1 \leq e_2)$	$e_1 - e_2 > 0$	消除否定并调整边界
整体符号规范化	$-e \leq 0$	$e \geq 0$	消除整体负号造成的差异

在子句层面, 系统将归一化后的文字加入有序集合, 从而消除重复文字, 并依据项结构的内部全序生成稳定文字列表。这里的稳定顺序不表示文字具有语义优先级, 而是保证同一文字集合在不同构造路径下具有一致的内部表示。

4.2 基于抽象语法树节点数的文字删除策略

对于进入排序阶段的每个文字, 算法 1 第 6 行计算 $a(l) = \text{ASTNodes}(l)$, 并将其作为排序键的一部分。规范化能够消除部分表达差异, 但不同文字所包含的约束结构仍存在明显区别。本文使用

抽象语法树节点数衡量文字的语法结构规模。设文字对应项 t 的节点数为 $A(t)$ ，其定义如下：

$$A(t) = \begin{cases} 1, & t \text{ 为变量或常量}, \\ 1 + \sum_{i=1}^n A(t_i), & t = f(t_1, t_2, \dots, t_n). \end{cases} \quad (10)$$

其中，函数符号、算术运算符、比较符号及逻辑运算符均作为抽象语法树内部节点参与计数。因此，文字 $x > 0$ 的节点数低于 $x + y - z \geq 3$ ，后者包含更多运算节点和变量项。

在组合排序策略中，当多个文字具有相同的历史反馈权重时，优先尝试删除抽象语法树节点数较低的文字。该设计基于一个启发式假设：结构较简单的文字通常描述单变量边界或局部状态条件，而结构较复杂的文字可能包含更多变量关联。需要强调的是，抽象语法树节点数仅是结构复杂度的低成本代理指标，并不直接等价于跨变量耦合强度；复杂的单变量表达式同样可能具有较大的节点数。因此，本文仅将该指标作为历史权重相同时的辅助判据，并通过消融实验考察其实际作用。

4.3 基于历史反馈权重的自适应删除策略

在算法 1 第 8 行的排序键 $\langle d(l), w(l), a(l), p(l) \rangle$ 中， $w(l)$ 来自历史反馈权重。仅依赖静态语法结构无法反映文字在实际验证过程中的作用。随着 IC3 不断处理新的归纳反例，某些文字可能反复出现在成功概括后的子句中，这表明它们更可能是维持初始性或相对归纳性的关键条件。为利用此类历史信息，本文为每个文字维护动态反馈权重。

权重表在 IC3QE 分析进程启动时初始化为空表，并在该进程内的归纳概括调用之间持续维护；未出现文字的权重按 0 处理。设第 k 次成功概括后文字 l 的权重为 $w_k(l)$ 。在生成新的概括子句 C_{gen} 时，权重按照下式更新：

$$w_{k+1}(l) = \begin{cases} \alpha w_k(l) + 1, & l \in \text{Lits}(C_{\text{gen}}), \\ \alpha w_k(l), & l \notin \text{Lits}(C_{\text{gen}}). \end{cases} \quad (11)$$

其中， $0 < \alpha < 1$ 为衰减因子，当前实现取 0.99。增量项表示该文字再次出现在成功概括结果中；衰减项用于逐步降低早期历史的影响，使权重能够反映近期验证阶段的文字保留特征。代码仅在归纳概括确实删除至少一个文字并生成新子句时执行该更新。

在后续归纳概括中，历史权重较低的文字优先尝试删除，历史权重较高的文字延后处理。这样可以减少对高频保留文字的重复无效删除检查，同时优先探索历史概括结果中保留频率较低的文字。综合考虑不同特征后，本文采用表 3 所示的字典式优先级确定文字删除顺序。

表 3: 自适应文字删除的排序优先级

优先级	排序依据	删除顺序
1	是否包含除法或整数除法运算	包含此类运算的文字优先
2	历史反馈权重	权重较低的文字优先
3	抽象语法树节点数	节点数较低的文字优先
4	原始位置	保持文字原有的相对顺序

算法 1 第 8 行按照表 3 所示的优先级完成字典序升序排序。 $d(l)$ 是字典序排序键的第一维主键；通过为含除法或整数除法算子的文字赋予较低键值 0，并为其他文字赋值 1，此类文字将被优

先扫描并尝试删除，以便在其可被安全移除时尽早简化后续 SMT 查询。历史反馈权重构成第二维排序依据，抽象语法树节点数作为权重相同时的辅助判据，原始位置则用于保证排序稳定性。

需要统一说明的是，上述结构特征和历史反馈信息仅用于确定文字的尝试顺序，并不直接决定文字能否被删除。算法 1 第 13 行从当前子句 C_g 中删除文字 l ，得到候选子句 $C_{\text{cand}} = C_g \setminus \{l\}$ 。无论当前文字因何种排序依据被选中，候选子句都必须首先通过算法第 14 行的初始性检查：

$$\text{SAT}(I \wedge \neg C_{\text{cand}}) = \text{false}, \quad (12)$$

同时还需要通过算法第 16 行相对于前驱帧的归纳性检查：

$$\text{SAT}(P \wedge F_{i-1} \wedge C_{\text{cand}} \wedge T \wedge \neg C'_{\text{cand}}) = \text{false}. \quad (13)$$

只有式 (12) 和式 (13) 对应的查询均不可满足时，算法才在第 18 行接受本次删除；否则保留文字 l 并继续处理后续文字。由此可见，本文方法改变的是归纳概括的搜索路径，而不是 IC3 原有的逻辑判定条件。

其中， C'_{cand} 表示候选子句在下一状态变量上的形式。若初始性查询可满足，说明候选子句会排除合法初始状态；若相对归纳性查询可满足，说明候选子句无法在当前帧和转移关系下保持。上述任一情况出现时，当前文字均不能删除。只有两个查询均返回不可满足时，算法才接受本次删除。同时，算法禁止将阻断子句概括为空子句。

完成概括后，仅当确实生成了更小的新子句时，算法才在第 21 行对已有权重执行衰减，并在第 23 行增加概括结果中保留文字的权重。由此，历史反馈只影响后续搜索路径，最终概括结果仍由 IC3 原有的逻辑检查决定。

命题 1 (排序变换的逻辑保持性). 设 π 为待尝试文字集合上的任意排列。若算法仅在式 (12) 和式 (13) 对应的查询均不可满足时接受一次文字删除，则算法在顺序 π 下返回的子句 C_g 满足初始性与相对归纳性。文字排列只可能改变最终到达的局部最小子句，不会改变单次删除的逻辑接受条件。

证明. 初始子句 C 在进入归纳概括前已经满足初始性与相对归纳性。算法每次接受删除时，均通过两项求解器查询验证候选子句 C_{cand} 满足相同条件，随后才令 $C_g \leftarrow C_{\text{cand}}$ ；若任一查询可满足，则保留当前文字， C_g 不发生变化。因此，对接受删除的次数作归纳可知，循环过程中及算法返回时的 C_g 始终满足两项条件。排列 π 仅改变候选子句被访问的顺序，不参与上述判定，故不影响所接受子句的逻辑有效性。 $\square$

4.4 实现细节与开销分析

本文方法在 Kind2 的 IC3QE 流程中实现。等式和不等式规范化分别由 `--ic3qe_ge_eq_can` 与 `--ic3qe_ge_ineq_can` 控制，二者作用于 `QE.generalize` 返回的文字列表，默认均关闭。文字频率排序由 `--ic3qe_freq_sort` 启用；抽象语法树节点数由 `--ic3qe_ast_complexity` 启用，并仅在频率排序开启时作为历史权重相同情况下的次级判据。上述两个排序开关同样默认关闭。删除顺序调整位于阻断子句的归纳概括函数中，仅当前驱帧非空且文字数大于 1 时执行稳定排序。该实现复用 Kind2 中已有的项结构、项集合和 SMT 激活文字机制，不需要修改求解器接口，也不改变帧序列、证明义务或子句传播的语义。

从时间开销看，设待概括子句包含 m 个文字，单个文字的语法树规模上界为 s ，当前权重表包含 $h = |\text{dom}(W)|$ 个键。规范化和抽象语法树节点数计算的开销与文字大小近似线性相关，排序开销为 $O(m \log m)$ ；若本次概括成功删除文字，则全表衰减和保留文字增量更新的开销为 $O(h + m)$ 。因此，不计 SMT 查询时，单次成功概括引入的额外开销上界为 $O(ms + m \log m + h)$ 。

相比之下，归纳概括的主要成本通常来自初始性和相对归纳性 **SMT** 查询，每次尝试删除一个文字最多触发两次查询。

需要指出的是，启用相应规范化开关时，历史反馈权重以规范化后的文字为键进行维护；未启用时则直接使用原始项作为键。等式规范化会显式跳过含除法、整数除法、量词或 **let** 的表达式。不等式规范化仅在外层比较能够被线性形式收集过程处理时执行，否则保留原文字。上述处理边界与当前 **Kind2** 实现保持一致。

5 实验设计

为了评估本文方法的有效性，实验围绕三个问题展开：

1. 与 **Kind2** 原始 **IC3QE** 删除顺序相比，自适应排序能否降低总体求解时间或求解器调用次数？
2. 归一化、除法优先、历史反馈权重和抽象语法树节点数分别贡献了多少性能收益？
3. 该方法是否会改变已求解实例集合，或者在部分实例上引入明显退化？

实验对象包括 **Kind2** 自带回归用例以及公开 **Lustre/SMT** 安全性验证基准中包含线性整数算术的实例。每个实例在相同硬件、相同 **SMT** 求解器和相同超时限制下运行，记录验证结果、总运行时间、**IC3** 主循环时间、**SMT** 查询次数、归纳概括调用次数、成功删除文字数、学习子句平均长度以及最大帧深度等指标。对于运行时间较短或波动较大的实例，实验重复运行多次并报告中位数，以减少系统噪声对结论的影响。

表 4 给出了消融配置。**Baseline** 表示原始文字顺序；**Norm** 仅启用算术文字归一化；**Freq** 使用历史反馈权重；**Freq+AST** 在历史权重相同的情况下加入抽象语法树节点数；**Full** 进一步加入除法或整数除法优先规则。通过这些配置可以区分“表达式表示更稳定”与“删除顺序更有效”两类因素。

表 4: 实验消融配置

配置	归一化	除法优先	历史权重	AST 节点数	说明
Baseline	-	-	-	-	Kind2 原始归纳概括删除顺序
Norm	是	-	-	-	仅考察规范表示对重复文字和历史键稳定性的影响
Freq	是	-	是	-	根据成功概括结果中的保留文字更新权重
Freq+AST	是	-	是	是	在权重相同的文字之间优先尝试简单结构
Full	是	是	是	是	本文完整排序策略

结果呈现同时包含总体统计和实例级散点图。总体统计报告 **solved** 数量、运行时间几何平均值、**SMT** 查询次数几何平均值和概括后子句平均长度；实例级散点图展示 **Full** 与 **Baseline** 在运行时间和 **SMT** 查询次数上的一一对比。若某些实例明显变慢，则进一步分析其文字类型、概括失败次数和学习子句长度变化，说明启发式排序在哪些结构上可能不适用。

6 局限性与威胁

本文方法属于启发式优化，其收益依赖于基准中算术文字的结构分布和 IC3 搜索过程产生的历史信息。对于以布尔结构为主、子句很短或归纳概括调用次数很少的实例，排序带来的收益可能有限；对于高度非线性或大量包含除法、量词的表达式，归一化会采取保守策略，因此不能保证所有等价约束都被统一。

另一个威胁来自历史反馈的阶段性的。高权重文字在早期可能确实是关键约束，但随着帧序列推进，其重要性可能下降。本文通过衰减因子降低早期历史的影响，但衰减率仍是经验参数。后续可考虑根据帧深度、最近若干次概括成功率或 SMT 查询结果动态调整衰减速度。

最后，本文排序策略不改变 IC3 的逻辑接受条件，因此不会破坏被接受子句的初始性和相对归纳性；但它可能改变搜索路径，从而改变最终学习到的子句集合。实验评价不能只报告运行时间，还需要报告求解结果一致性、超时实例变化和子句质量指标，以避免将偶然路径差异误判为稳定收益。

7 小结

本文针对 IC3/PDR 归纳概括中的文字删除顺序问题，提出了一种结合算术文字归一化、抽象语法树节点数和历史反馈权重的自适应排序方法。该方法的核心思想是在不改变初始性和相对归纳性判定条件的前提下，利用文字结构和近期概括结果改变贪心删除过程的访问顺序。对于线性整数算术场景，归一化有助于稳定历史键和结构度量，历史权重有助于延后反复保留的关键文字，抽象语法树节点数和除法优先规则则提供低开销的结构性辅助信号。后续实验需要通过系统消融和实例级分析进一步确认各个信号的贡献及适用边界。

参考文献

- [1] A. R. Bradley. SAT-Based Model Checking without Unrolling. In *Verification, Model Checking, and Abstract Interpretation*, pp. 70–87, 2011.
- [2] N. Eén, A. Mishchenko, and R. Brayton. Efficient Implementation of Property Directed Reachability. In *Formal Methods in Computer-Aided Design*, pp. 125–134, 2011.
- [3] Z. Hassan, A. R. Bradley, and F. Somenzi. Better Generalization in IC3. In *Formal Methods in Computer-Aided Design*, pp. 157–164, 2013. doi:10.1109/FMCAD.2013.6679405.
- [4] A. Champion, A. Mebsout, C. Sticksel, and C. Tinelli. The Kind 2 Model Checker. In *Computer Aided Verification*, 2016.
- [5] N. Halbwachs, P. Caspi, P. Raymond, and D. Pilaud. The Synchronous Data Flow Programming Language LUSTRE. *Proceedings of the IEEE*, 79(9):1305–1320, 1991.
- [6] R. Dureja, J. Li, G. Pu, M. Y. Vardi, and K. Y. Rozier. Intersection and Rotation of Assumption Literals Boosts Bug-Finding. In *Verified Software: Theories, Tools, and Experiments*, pp. 180–192, 2020.

- [7] Y. Xia, A. Becchi, A. Cimatti, A. Griggio, J. Li, and G. Pu. Searching for *i*-Good Lemmas to Accelerate Safety Model Checking. In *Computer Aided Verification*, pp. 288–308, 2023.
- [8] T. Seufert, F. Winterer, C. Scholl, K. Scheibler, T. Paxian, and B. Becker. Everything You Always Wanted to Know About Generalization of Proof Obligations in PDR. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2022. doi:10.1109/TCAD.2022.3198260.
- [9] Y. Dong, Y. Chen, J. Li, G. Pu, and O. Strichman. Revisiting Assumptions Ordering in CAR-Based Model Checking. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2025. doi:10.1109/TCAD.2025.3551658.